

Test 2 OOP

Service & → when I don't want copy (ex: Service & service in UI)

Car* → polymorphism / new

car → simple object

Default constructor → no params → Service S1;

Copy constructor → when I create obj. from another obj. → Service S3(S1);

Manually destroy (⇒) "new"

when creating: base class → derived class

destructing: derived class → base class.

auto → copies the element

auto& → modifies original

const auto& → does not copy or change

TERNARY OPERATOR:

std::string efficiency = d → isEfficient() ? "EF" : "IN"

std::sort(begin, end, comparator)

- sort

- find-if

- copy-if

- remove-if

- count-if

- any-of

- all-of

delete → destroy elem from memory

erase → eliminate elem from vector

• only erase ⇒ pointer removed from vector, but object remains in memory ⇒ memory leak.

• only delete ⇒ pointer remains in vector, but memory is freed

vector<Dwelling> ⇒ only erase.

for(auto s: sensors){

std::cout << s → toString();

(POINTER)

• delete/update

• aggrog...

• isalpha etc

• strcat etc

• const auto

vs

auto

for(const auto& d: dwellings){

std::cout << d.toString();

}

for(auto& d: dwellings){

d.setPrice(1000);

INHERITANCE: virtual, override, = 0, ...

AGGREGATION: class has pointer/vector to other objects, but doesn't necessarily own them.

COMPOSITION: class contains objects + owns them.

DEPENDENCY: class uses other class as param

const std::vector<double>& recordings

→ receive the vector without copy

no modif.

avoids copying

YYYY.MM.DD

SUBSTR

int year = std::stoi(date.substr(0,4));

int month = std::stoi(date.substr(5,2));

int day = std::stoi(date.substr(8,2));

std::string filename = "clients.txt"

std::string extension = filename.substr(filename.size() - 4);

CTIME

#include <ctime>

int currentYear() {

std::time_t now = std::time(nullptr);

std::tm* local = std::localtime(&now);

return local → tm_year + 1900;

"Ignore" after success → clears the end of the line

"Ignore after failure" → clears the wrong input (whole line)

"unique" ADD

for(auto s: sensors){

if(s → get...() == s.new → get...()) {

delete s;

throw std::runtime_error("Exists");

sensors.push_back(s);

catch(const std::exception& e) → normal errors

catch(...) → any errors, but does not specify what.

better readString:

std::string value;

while(true) {

std::cout << message;

std::getline(std::cin, value);

if(!value.empty()) return value;

std::cout << "Invalid";

MAIN:

```
Repository repo;
Service service {repo};
UI ui {service};
ui.run();
return 0;
```

ui:

```
readInteger (message) {
    int value; // double
    while (true) {
        std::cout << message;
        if (std::cin >> value) {
            std::cin.ignore (...);
            return value;
        }
        std::cout << "invalid ...";
        ... clear(); // take cin out of error state
        ... ignore (...); // throw away wrong input (while true)
    }
}

readString (message) {
    std::string value;
    std::cout << message;
    std::getline (std::cin, value);
    return value;
}

run() {
    while (true) {
        menu();
        int option = readInteger ("...");
        if (option == 0) break;
        switch (option) {
            case 1:
                ...
                break;
            case 2:
                ...
                break;
            default:
                std::cout << "Invalid ...";
        }
    }
}
```

display without copying
~~vector < Car >~~ => for (const auto& x: cars)
 vector < Car* > => for (auto x: cars)
 update => for (auto& c: s)
 real objects (original modif.)
 pointer alternative?
 for (auto* x: cars)

REPO/SERVICE

```
writelnFile (const ... filename) {
    std::ofstream fout (filename);
    if (!fout.is_open()) return;
    std::vector < Stuff* > stuffs = this->arepo.get();
    std::sort (stuffs.begin(), stuffs.end(),
        [](Stuff* a, Stuff* b) {
            return a->get ... () < b->get ... ();
        });
    for (auto s: stuffs) {
        (std::string ...)
        fout << s->toString() << '\n';
    }
    fout.close();
}
```

(a -> isResistorOK() ? "OK" : "NOT OK")

```
std::vector < double > recordings;
double x;
while (std::cin >> x && x != -1) {
    recordings.push_back(x);
}
```

```
std::sort (cars.begin(), cars.end(),
    [](Car* a, Car* b) {
        return a->get ... () < b->get ... ();
    });
```

Car* => pointer => "..."
 Car => object => "..."

vector < T* > delete: ↴

```
for (int i = 0; i < clients.size(); i++) {
    if (clients[i] -> getName() == name) {
        delete clients[i];
        clients.erase (clients.begin() + i);
        return;
    }
}
```

```
try {
    service.add (obj);
    std::cout << "Successful";
} catch (const std::exception& e) {
    delete obj; std::cout << e.what() ...
}
```

std::vector < Sensor* > sensors;
 => => Sensor* sensor; / Sensor sensor;